Ad_Click_Fraud_Detection_Using_Machine_Learning_and_Deep _Learning_Algorithm

ABSTRACT

With the exponential growth of mobile advertising, fraudulent ad clicks have become a major concern, leading to substantial financial losses for advertisers. This project, Ad Click Fraud Detection Using Machine Learning and Deep Learning Algorithms, aims to develop an intelligent and scalable detection system capable of distinguishing between genuine and fraudulent clicks. The dataset contains crucial features such as IP address, application ID, device type, operating system version, publisher channel ID, click timestamp, and attribution time (if the app was downloaded post-click). The target variable is_attributed indicates whether a click led to an actual app download.

A wide range of models were implemented, including traditional Machine Learning algorithms like Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Naive Bayes, SVM, Gradient Boosting, LightGBM, and XGBoost. Additionally, Deep Learning architectures such as Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Deep Neural Networks (DNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) were explored. A Stacking Classifier was used to combine model strengths and optimize performance. The backend integrates all models to compare results, while the Flask-based frontend allows users to input click features and receive real-time fraud detection outputs.

Keywords:

Ad Click Fraud, Machine Learning, Deep Learning, Flask App, Stacking Classifier, LSTM, XGBoost, Click Prediction, Mobile Advertising, Classification.

OBJECTIVE OF PROJECT:

The main objective of this project is to build an efficient and intelligent system capable of detecting fraudulent ad clicks in real time. By analyzing various features such as IP address, app ID, device type, OS version, channel ID, click time, and attribution status, the goal is to classify whether a click is genuine or potentially fraudulent. This system utilizes a range of Machine Learning and Deep Learning models—including Logistic Regression, Random Forest, XGBoost, CNN, LSTM, and more—to ensure high accuracy and generalization. The model performance is compared, and the best-performing models are integrated into a user-friendly Flask web application. This enables users to input features manually and receive instant predictions on click legitimacy.

PROBLEM STATEMENT:

In digital advertising, especially on mobile platforms, click fraud has emerged as a significant challenge. Fraudulent clicks—generated by bots or deceptive users—result in wasted advertising budgets, reduced ROI, and skewed analytics. Existing detection systems are often rule-based, static, or unable to generalize across diverse data. The problem addressed in this project is how to build a data-driven, intelligent, and adaptive system that can accurately classify ad clicks as fraudulent or genuine based on user behavior and device metadata. The solution must be scalable, robust, and deployable in real-time environments. The project seeks to use a variety of ML/DL models and compare their effectiveness in solving this issue while offering a deployable interface via a Flask web application.

MOTIVATION:

Click fraud is a growing issue in the mobile advertising industry, where fraudulent actors simulate fake clicks to earn revenue unfairly. This not only impacts advertisers financially but also skews campaign performance metrics and undermines trust in digital marketing platforms. The motivation behind this project is to create a reliable system that helps advertisers and marketers detect fraudulent clicks accurately, thereby preventing unnecessary ad spend and improving campaign efficiency. By leveraging both machine learning and deep learning models, and deploying the solution with an easy-to-use interface, the project aims to bridge the gap between complex predictive analytics and practical usage. It empowers stakeholders with technology to mitigate digital ad fraud in an effective, automated manner

SCOPE:

This project addresses the issue of click fraud in mobile advertising, focusing on the detection of fraudulent clicks before app attribution. It supports analysis and prediction using a combination of traditional ML models (e.g., Logistic Regression, Random Forest, XGBoost) and advanced deep learning architectures (e.g., CNN, LSTM, GRU). The solution includes backend training, evaluation of multiple models, and frontend deployment using Flask. The scope extends to model comparison, real-time fraud prediction, and system deployment in real-world advertising platforms. Future extensions may involve auto-updating models with live data, scaling for large-volume detection, and integration with ad networks or APIs to automate fraud prevention mechanisms efficiently.

PROJECT INTRODUCTION:

In today's digital ecosystem, mobile advertising plays a crucial role in reaching targeted audiences efficiently. With the increasing shift toward app-based promotions and pay-per-click models, advertisers rely heavily on the accuracy and legitimacy of user interactions, especially ad clicks. However, the rise of click fraud—where bots or malicious users generate fake clicks—has become a significant threat. These fraudulent activities not only waste advertising budgets but also distort campaign performance metrics, leading to poor strategic decisions and reduced return on investment (ROI).

To address this growing issue, our project titled "Ad Click Fraud Detection Using Machine Learning and Deep Learning Algorithms" aims to build an intelligent fraud detection system that can automatically classify ad clicks as either genuine or fraudulent. The system leverages a dataset containing key features such as IP address, app ID, device type, operating system version, channel ID, click timestamp, attributed download time, and a target label is_attributed which indicates whether a click resulted in a legitimate app download.

A wide range of algorithms is employed to ensure robustness and accuracy. Classical machine learning models like Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Naive Bayes, SVM, Gradient Boosting, LightGBM, and XGBoost are implemented. In parallel, deep learning techniques such as Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and Deep Neural Networks (DNN) are trained to capture hidden patterns and sequential behaviors in the data.

To further enhance predictive performance, a Stacking Classifier is applied, which combines the strengths of multiple models and produces a meta-classifier capable of improved generalization. Each model's performance is evaluated using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. The best-performing models are selected for integration.

The backend system manages data preprocessing, model training, and predictions, while the Flask web application serves as the frontend. Users can input ad click features via the interface and receive immediate feedback on whether the click is likely to be fraudulent. This end-to-end solution not only supports data-driven fraud detection but also offers real-time deployment capabilities suitable for integration with live ad networks and fraud prevention systems.

SYSTEM ANALYSIS

1. Existing System

The existing systems for ad click fraud detection rely on traditional Machine Learning and basic Deep Learning algorithms. Models like Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Naive Bayes, and Support Vector Machine (SVM) are commonly used due to their simplicity and interpretability. Additionally, advanced techniques such as Gradient Boosting (GB), LightGBM, and XGBoost offer improved accuracy through ensemble learning. In the realm of deep learning, Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Deep Neural Networks (DNN), and Recurrent Neural Networks (RNN) have been explored to capture non-linear and sequential patterns.While these methods provide reasonable performance, they often operate independently and struggle with complex temporal dependencies or require high tuning efforts. Moreover, they are limited in real-time adaptability and robustness when deployed on dynamic, large-scale advertising platforms.

Disadvantages of the Existing System

- Inability to capture long-term temporal dependencies (especially in RNN).
- High risk of overfitting on imbalanced data.
- Lower accuracy when applied to noisy or high-dimensional features.

- Difficult to combine strengths of multiple models effectively.
- Lack of scalability and generalization for unseen fraud patterns.
- Limited real-time prediction capabilities in deployment environments.

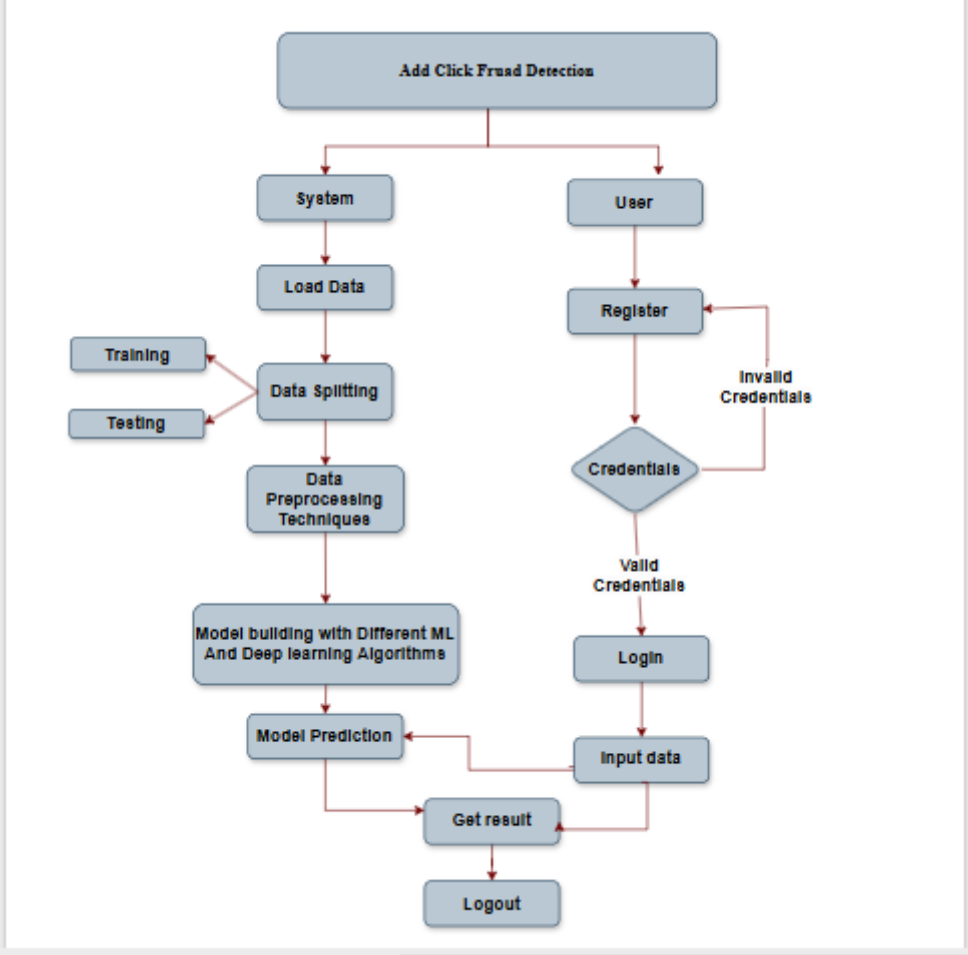
2. Proposed System

To overcome the shortcomings of the existing models, the proposed system incorporates a hybrid approach using Stacking Classifier, LSTM, and GRU networks. The Stacking Classifier strategically combines multiple base learners (both ML and DL) to enhance generalization and predictive power through meta-learning. Simultaneously, Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures are employed to learn sequential dependencies and capture time-based fraud patterns more effectively than standard RNNs. This combination ensures that both static features (like IP, device) and temporal features (like click time) are intelligently processed.The backend performs model training, cross-validation, and prediction generation, while a Flask-based frontend enables real-time fraud detection through user inputs. The proposed system thus ensures higher accuracy, robustness, and adaptability for detecting complex fraud behaviors in large-scale ad environments.

Advantages of the Proposed System

- Captures complex temporal behavior using LSTM and GRU.
- Improves overall prediction accuracy with stacked ensemble learning.
- Reduces overfitting through model diversity and regularization.
- Adapts well to evolving fraud patterns and large datasets.
- Supports real-time predictions via an intuitive web interface.
- Provides scalable and robust architecture suitable for deployment.

PROJECT FLOW



HARDWARE & SOFTWARE REQUIREMENTS

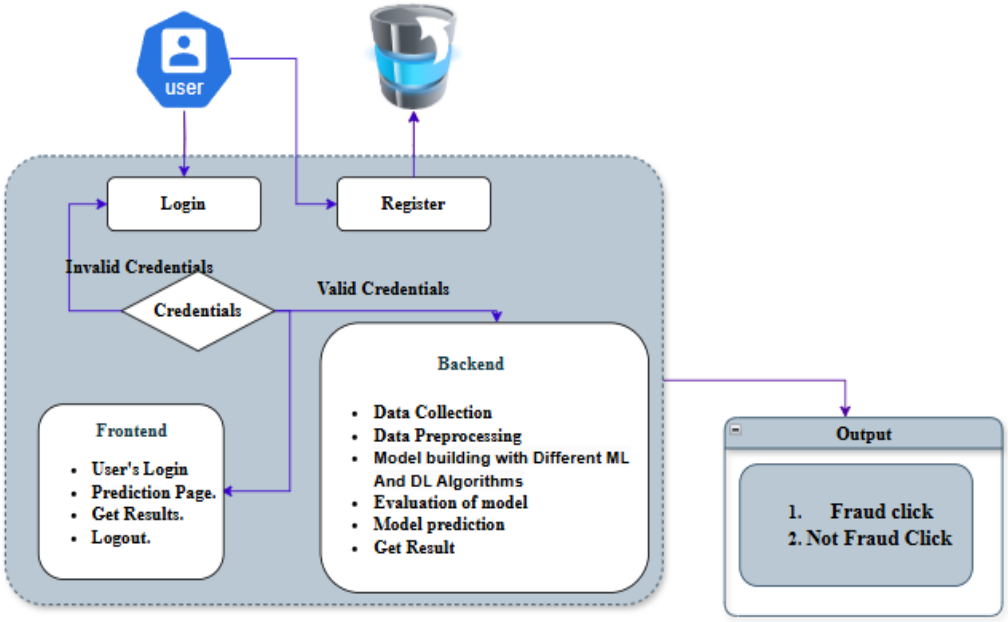
1. SOFTWARE REQUIREMENS

Operating System	: Windows 7/8/10
Server side Script	: HTML, CSS, Bootstrap & JS
Programming Language	: Python
Libraries	Flask, Pandas,Tensorflow, Sklearn, Numpy , Seaborn
IDE/Workbench	: VSCode
Technology	: Python 3.8+
Server Deployment	: Xampp Server
Database	: MySQL

2. HARDWARE REQUIREMENTS

Processor	- I3/Intel Processor
RAM	- 8GB (min)
Hard Disk	- 128 GB
Key Board	- Standard Windows Keyboard
Mouse	- Two or Three Button Mouse
Monitor	- Any

3. ARCHITECTURE:



🔧 System Modules

1. Data Collection:

This module is responsible for collecting ad click data from available datasets or live sources. The dataset includes essential features such as IP address, app ID, device type, OS version, channel ID, click time, and attribution time. The target variable indicates if the user downloaded the app post-click.

2. Data Preprocessing:

Raw data is often noisy or incomplete. This module handles missing values, encodes categorical variables, and scales numerical features. Timestamp fields like click_time are converted into meaningful features (hour, day, etc.) to improve model learning.

3. **Splitting Data:**

   The dataset is split into training and testing sets to evaluate model generalization. Stratified splitting ensures class balance is maintained, which is especially important for imbalanced data like fraud detection.

4. **Model Training:**

   Multiple models including Logistic Regression, Decision Tree, Random Forest, KNN, ANN, SVM, XGBoost, LightGBM, CNN, RNN, LSTM, GRU, and Stacking Classifier are trained on the prepared training dataset using appropriate hyperparameters.

5. **Model Evaluation:**

   Each trained model is evaluated using metrics like accuracy, precision, recall, F1-score, and ROC-AUC. This module compares model performance to identify the best candidates for deployment.

6. **Model Prediction:**

   This module handles real-time predictions. It receives input features from users and applies the best-performing model(s) to predict whether a given ad click is fraudulent or legitimate.

👤 User Modules

1. **Login Page:**

Allows registered users to securely log into the system using their email and password. Session management ensures secure and authenticated access to other parts of the application.

2. **Registration Page:**

New users can create an account by filling in required details like name, email, and password. Upon successful registration, users are redirected to the login page.

3. **Upload Page:**

Users can upload new datasets for training or testing purposes. The uploaded file is validated for format and structure before being processed for modeling.

4. **Model Page:**
   This page displays the performance of all trained models. Users can view evaluation as accuracy model for prediction tasks.

5. **Prediction Page:**
   Provides an interface where users input features such as IP, app ID, device type, OS, channel, and click time. The model uses these inputs to predict whether the click is **fraudulent or not** and displays the result instantly.

6. **Logout**
   To ensure data security and session integrity, the logout module allows users to safely end their session. When the logout option is selected, the system clears all active session data and redirects the user to the login page.